Cuestionario: SQL Data Warehouse from Scratch

P8 · Data Warehouse from Scratch · Data with Baraa · Arquitectura Medallion en SQL Server

Integrantes del equipo	Isaac Pancardo Botello Carlos Emiliano Mendoza Hernandez Alan Magno Martinez Muñoz
Liga al GitHub	https://github.com/botellopai18/data-warehouse-mineria-datos
Fecha	15 de octubre de 2026

A. Opción múltiple (10 puntos)

1. ¿Cómo se materializan los objetos de la capa Gold?

a) Tablas físicas con índices
(b) Vistas ✔
c) Tablas temporales
d) Vistas indexadas

Justificación: en `scripts/gold/ddl_gold.sql` los tres objetos se crean con `CREATE VIEW` (gold.dim_customers, gold.dim_products, gold.fact_sales); Gold no tiene proceso de carga.

2. ¿Cómo se genera la llave sustituta de dim_customers?

a) IDENTITY(1,1)
b) NEWID()
(c) ROW_NUMBER() OVER (ORDER BY cst_id) ✔
d) Hash de cst_key

Justificación: `ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key` dentro de la vista.

3. ¿Qué tipo de historización tienen las dimensiones?

a) SCD tipo 2
(b) Ninguna: solo el dato más reciente ✔
c) SCD tipo 3
d) Snapshots mensuales

Justificación: el requerimiento del proyecto pide solo el dato más reciente. En Silver se conserva el último registro de cada cliente y dim_products filtra `WHERE prd_end_dt IS NULL` (elimina el historial).

4. Las columnas técnicas que agrega el warehouse (por ejemplo, la fecha de carga) llevan el prefijo:

a) etl_
(b) dwh_ ✔
c) meta_
d) sys_

Justificación: `dwh_create_date DATETIME2 DEFAULT GETDATE()` en todas las tablas Silver, según las convenciones de nombres.

B. Respuesta corta (10 puntos)

5. En crm_cust_info hay cst_id duplicados. ¿Con qué criterio se conserva un registro y con qué función?

Se conserva el **registro más reciente** de cada cliente, es decir, el de **mayor cst_create_date**. Se usa la función de ventana **ROW_NUMBER()** particionando por cst_id y ordenando por fecha de creación descendente; se queda la fila con `flag_last = 1`. Además se descartan los registros con cst_id NULL.

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY cst_id
                                 ORDER BY cst_create_date DESC) AS flag_last
    FROM bronze.crm_cust_info
    WHERE cst_id IS NOT NULL
) t
WHERE flag_last = 1;
```

6. ¿Por qué se quita el prefijo NAS en erp_cust_az12 y el guion en erp_loc_a101?

Para que sus llaves **coincidan con cst_key del CRM** y se puedan integrar (JOIN) en gold.dim_customers. En el CRM la llave es `AW00011000`; en el ERP llega como `NASAW00011000` (AZ12) y `AW-00011000` (LOC_A101). Sin la limpieza, los LEFT JOIN no encontrarían coincidencias y fecha de nacimiento, género (ERP) y país quedarían en NULL.

```
-- erp_cust_az12
CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid)) ELSE cid END AS cid
-- erp_loc_a101
REPLACE(cid, '-', '') AS cid
```

C. Analiza la afirmación (15 puntos)

7. Explica por qué se decidió modelar Gold como copo de nieve.

Premisa falsa. En el video la capa Gold **no** se modela como copo de nieve, sino como **esquema estrella (star schema)**: una tabla de hechos (fact_sales) rodeada de dimensiones desnormalizadas (dim_customers, dim_products) que se unen directamente a ella.

Se eligió estrella porque es más simple de entender para usuarios de negocio, requiere menos JOINs y es la estructura que mejor funciona con herramientas de BI y reportes. Por eso las dimensiones se desnormalizaron: dim_customers integra crm_cust_info + erp_cust_az12 + erp_loc_a101, y dim_products integra crm_prd_info con su categoría de erp_px_cat_g1v2, en lugar de dejar la categoría como una dimensión separada (que sería copo de nieve).

8. ¿Qué ventaja tuvo unir fact_sales con dim_products usando prd_id?

Premisa falsa. fact_sales **no** se une con dim_products por prd_id; crm_sales_details ni siquiera tiene esa columna. La unión se hace por la llave de negocio del producto: `sd.sls_prd_key = pr.product_number`, donde product_number es el prd_key que en Silver se derivó quitando los primeros caracteres de categoría (`SUBSTRING(prd_key, 7, LEN(prd_key))`), justamente para que coincidiera con sls_prd_key.

Ese JOIN se usa para **reemplazar la llave de negocio por la llave sustituta product_key** en la tabla de hechos (igual que `sls_cust_id = customer_id` → customer_key). La ventaja es que la fact queda conectada a las dimensiones por llaves sustitutas propias del warehouse, independientes de los códigos de los sistemas fuente. Como dim_products solo tiene productos vigentes (prd_end_dt IS NULL), cada product_number aparece una sola vez y el JOIN no duplica ventas.

```
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products  pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id
```

D. Aplicación (45 puntos)

9. Aplica las reglas de Silver a los registros de bronze.crm_cust_info y escribe la salida. ¿Qué género tendría Lauren en Gold si en el ERP aparece como 'Female'?

Reglas de silver.load_silver para esta tabla: (1) excluir cst_id NULL; (2) quedarse con el registro más reciente por cst_id; (3) TRIM a los nombres; (4) `UPPER(TRIM())` y mapear género F→Female, M→Male, otro→'n/a'; (5) estado civil S→Single, M→Married, otro→'n/a'; (6) dwh_create_date se llena con GETDATE().

Fila bronze	¿Qué pasa?
29466 · ' Lauren ' · 'f' · 'M' · 2026-01-25	**Se descarta**: es un duplicado más antiguo (flag_last = 2). Aunque traía género 'f', se pierde.
29466 · 'Lauren' · NULL · 's' · 2026-01-27	**Se conserva**: es el registro más reciente (flag_last = 1).
NULL · 'Ana' · 'F' · 'S' · 2026-02-01	**Se descarta**: cst_id es NULL (filtro WHERE cst_id IS NOT NULL).

Salida en silver.crm_cust_info (una sola fila):

cst_id	cst_key	cst_firstname	cst_gndr	cst_marital_status	cst_create_date	dwh_create_date
29466	AW00029466	Lauren	n/a	Single	2026-01-27	fecha/hora de carga (GETDATE())

cst_gndr = 'n/a' porque el valor es NULL (cae en el ELSE); cst_marital_status = 'Single' porque UPPER(TRIM('s')) = 'S'.

Género en Gold: Female. En dim_customers el CRM es la fuente maestra del género, pero solo si su valor es distinto de 'n/a'. Como el registro que sobrevivió quedó en 'n/a', se usa el valor del ERP (erp_cust_az12, ya normalizado en Silver a 'Female'):

```
CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr    -- CRM es la fuente maestra
     ELSE COALESCE(ca.gen, 'n/a')                  -- respaldo: ERP
END AS gender                                      -- 'n/a' -> 'Female'
```

10. Registro de ventas: sls_quantity = 2, sls_price = −50, sls_sales = NULL. ¿Qué sls_sales y sls_price quedan en Silver según el código del video?

```
CASE WHEN sls_sales IS NULL OR sls_sales <= 0
          OR sls_sales != sls_quantity * ABS(sls_price)
     THEN sls_quantity * ABS(sls_price)
     ELSE sls_sales END AS sls_sales,
CASE WHEN sls_price IS NULL OR sls_price <= 0
     THEN sls_sales / NULLIF(sls_quantity, 0)
     ELSE sls_price END AS sls_price
```

Columna	Evaluación	Resultado en Silver
sls_sales	sls_sales es NULL → se recalcula: 2 × ABS(−50) = 2 × 50	**100**
sls_price	sls_price = −50 ≤ 0 → se deriva: sls_sales / NULLIF(2, 0). Pero en un SELECT todas las expresiones leen las columnas **de origen** (bronze), no el sls_sales recién calculado; el sls_sales original es NULL, y NULL / 2 = NULL	**NULL**

Es decir, quedan **sls_sales = 100** y **sls_price = NULL**. Es una debilidad del código del video para este caso (sales nulo y precio negativo a la vez): se esperaría 50. Se corrige derivando el precio como `ABS(sls_price)` cuando es negativo, o calculando sls_sales primero en un CTE/subconsulta y derivando el precio a partir de ese valor ya corregido.

11. Una fecha de orden llega como 20250230. ¿Qué pasa al ejecutar silver.load_silver?

```
CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
     ELSE CAST(CAST(sls_order_dt AS VARCHAR) AS DATE)
END AS sls_order_dt
```

1. La regla del video solo valida que la fecha no sea 0 y que tenga 8 dígitos. 20250230 no es 0 y tiene 8 dígitos, así que **no se convierte en NULL**: entra al ELSE.

2. `CAST('20250230' AS DATE)` falla porque el 30 de febrero no existe: SQL Server lanza **Msg 241 – "Conversion failed when converting date and/or time from character string"**. El INSERT de crm_sales_details falla completo (no se inserta ninguna fila).

3. El error lo atrapa el **BEGIN TRY / BEGIN CATCH** del procedimiento, que imprime 'ERROR OCCURRED DURING LOADING SILVER LAYER', el mensaje, el número (241) y el estado del error. El procedimiento no

"truena" con error no controlado, pero la carga se detiene ahí.

4. Consecuencias: silver.crm_cust_info y silver.crm_prd_info sí se cargaron; **silver.crm_sales_details queda vacía** porque se truncó justo antes del INSERT y no hay transacción que haga rollback; y las tablas ERP (erp_cust_az12, erp_loc_a101, erp_px_cat_g1v2) **no se recargan** (conservan los datos de la corrida anterior). Como Gold son vistas, gold.fact_sales quedaría vacía.

5. Los quality checks del video (fechas <= 0, longitud != 8, fuera del rango 19000101–20500101) tampoco detectan este valor. Corrección propuesta: usar `TRY_CAST(CAST(sls_order_dt AS VARCHAR) AS DATE)`, que devuelve NULL en lugar de error, y agregar un check `WHERE TRY_CAST(CAST(sls_order_dt AS VARCHAR) AS DATE) IS NULL AND sls_order_dt <> 0`.